Guía completa: colisiones en videojuegos con Godot

Estructura de Datos II (IST 4031) · Universidad del Norte · 2026-30 · Integrantes: María Yusti, Axel Tornay, Juan Vizcaíno, Isac Monterrosa

Cómo leer esta guía. Está escrita para estudiar desde cero y defender el tema ante el profesor. Sigue el orden de la presentación. **A** fundamentos · **B** conceptos técnicos · **C** relación con la clase · **D** implementación real en Godot · **E** preguntas probables.
Marcas de origen: `[CLASE]` = viene de las diapositivas del curso (Angulo Madrid, s. f.-a a s. f.-e); `[EXTERNO]` = viene de fuentes externas (ver referencias); `[PROYECTO]` = es lo que realmente contiene la carpeta de Godot del grupo.

A. Fundamentos desde cero

A.1 ¿Qué es una colisión y por qué importa?

Un videojuego es una simulación que se repite muchas veces por segundo. En cada repetición (cuadro) los objetos se mueven. Una **colisión** ocurre cuando dos objetos ocupan, al mismo tiempo, una parte del mismo espacio. Sin colisiones, el personaje atravesaría el suelo, las pelotas pasarían a través de las paredes y no se podrían recoger monedas. Las colisiones son la base de casi toda la interacción de un juego.

A.2 Problemas que aparecen

- **Costo:** con muchos objetos, revisar todos contra todos es demasiado lento (diapositiva 10).
- **Exactitud:** las formas reales son complejas; hay que representarlas con formas simples.
- **Túnel (tunneling):** un objeto rápido puede estar a un lado de una pared en un cuadro y al otro lado en el siguiente, sin solaparse nunca en ningún cuadro [EXTERNO: Godot Engine contributors, s. f.-b; Ericson, 2004].
- **Respuesta:** detectar el choque no dice qué hacer; hay que decidir si rebota, se detiene, se desliza o dispara un evento.

A.3 Cómo se resuelve en general

Se divide en tres pasos que se repiten en cada cuadro de física:

Paso	Pregunta que responde	Qué produce
1. Fase amplia (broad phase)	¿Qué pares de objetos *podrían* chocar?	Una lista corta de pares candidatos
2. Fase estrecha (narrow phase)	De esos pares, ¿cuáles chocan de verdad y cómo?	Un *contacto*: normal y profundidad (o ninguno)
3. Respuesta	¿Qué hacemos con el contacto?	Separar, rebotar, deslizar, emitir una señal…

A.4 ¿Qué pasa internamente en cada cuadro?

Godot ejecuta la física en un paso fijo (60 veces por segundo por defecto) en `_physics_process`. En ese paso el motor mueve los cuerpos, busca contactos y aplica la respuesta o avisa al código mediante señales. Si ese proceso tardara más de unos 16,7 ms (1/60 s), el juego se vería lento; por eso importa que la detección sea barata.

A.5 Detección y respuesta: no son lo mismo

Detección = saber que hay contacto y medirlo. **Respuesta** = lo que el juego hace al respecto. Ejemplo: una moneda *detecta* que entró el jugador (no bloquea nada); una pelota *responde* rebotando. En Godot, un `Area2D` solo detecta, un `RigidBody2D` detecta y el motor responde, y un `CharacterBody2D` detecta pero la respuesta la programa quien hace el juego.

A.6 Tipos de colisión (por forma y por tiempo)

Criterio	Tipos	Idea
Por forma	Círculo, caja alineada a los ejes (AABB), polígono convexo	Cuanto más simple, más barata la prueba (diap. 6)
Por tiempo	Discreta / continua (CCD)	Discreta: se revisa en cada cuadro (puede haber túnel). Continua: se revisa la trayectoria entre cuadros [EXTERNO]
Por fase	Amplia / estrecha	Filtrar rápido, luego medir con exactitud

B. Conceptos técnicos (primero qué es, luego para qué sirve)

Término	Qué es	Para qué sirve
Forma de colisión	Figura geométrica simple que representa al objeto (en Godot, un recurso `Shape2D` dentro de un `CollisionShape2D`).	Hace la prueba barata y fiable; el dibujo del objeto puede ser más complejo.
AABB	*Axis-Aligned Bounding Box*: rectángulo cuyos lados son paralelos a los ejes X e Y.	Es la forma más barata; se usa como filtro en la fase amplia.
Polígono convexo	Polígono donde el segmento entre cualesquiera dos puntos queda dentro.	Es lo que SAT sabe manejar; una forma cóncava se parte en piezas convexas.
Normal de contacto	Vector unitario perpendicular al contacto, que indica la dirección de separación.	Para saber hacia dónde rebotar o empujar.
Profundidad de penetración	Cuánto se solapan los objetos a lo largo de la normal.	Para saber cuánto separarlos.
MTV	*Minimum Translation Vector*: normal × profundidad mínima.	El empujón más pequeño que separa los objetos.
Fase amplia	Etapa que descarta rápido los pares imposibles.	Evitar el costo $O(n^2)$.
Fase estrecha	Etapa que prueba con exactitud los pares candidatos.	Obtener el contacto (normal y profundidad).
SAT	Teorema del eje separador: dos convexos *no* chocan si existe un eje donde sus proyecciones no se solapan.	Prueba exacta para polígonos convexos.
Proyección	Sombra de la figura sobre un eje: un intervalo [mín, máx] obtenido con el producto punto de cada vértice con el eje.	Reduce un problema 2D a comparar intervalos 1D.
Túnel (tunneling)	Cruce de un objeto rápido a través de otro sin solaparse en ningún cuadro.	Explica por qué existe la detección continua.
Capa y máscara	`collision_layer`: en qué capa está el objeto. `collision_mask`: qué capas escanea.	Decidir, sin código, quién puede chocar con quién.
Señal (signal)	Mecanismo de Godot para avisar de un evento (por ejemplo `body_entered`).	Desacoplar: el objeto avisa, otro script reacciona.
Paso de física fijo	La física corre a una frecuencia fija (60 Hz por defecto), separada del dibujo.	Resultados estables sin importar los FPS.

B.1 Pruebas de la fase estrecha, paso a paso

Círculo–círculo. d = cB − cA. Hay colisión si $|d|^2 < (rA + rB)^2$. Se compara el cuadrado para evitar la raíz. Profundidad = (rA + rB) − |d|. *Traza:* radios 10 y 10, centros a distancia 15: 15 < 20 → colisión, profundidad 5, normal (1, 0). Costo O(1) en tiempo y espacio.

AABB–AABB. Se calcula el solape en cada eje: ox = min(finA, finB) − max(inicioA, inicioB), igual para oy. Hay colisión solo si ox > 0 *y* oy > 0. El eje de menor solape define la normal. *Traza:* cajas de 20×20; A en (−10,−10) y B en (5,−5): ox = 5, oy = 15 → colisión, profundidad 5, normal (1, 0). Costo O(1).

Círculo–AABB. Se busca el punto P de la caja más cercano al centro (recortando cada coordenada al rango de la caja, *clamp*). Hay colisión si la distancia del centro a P es menor que el radio. Caso especial: si el centro está dentro de la caja, P coincide con el centro y se sale por la cara más cercana. *Traza:* centro (17,0), radio 8, caja $[-10,10]^2$: P = (10,0), distancia 7 < 8 → profundidad 1, normal (−1, 0).

SAT (polígonos convexos). Para cada arista de ambos polígonos: (1) se toma el eje perpendicular a la arista; (2) se proyectan los dos polígonos; (3) si los intervalos no se solapan → no hay colisión y se termina; (4) si no, se guarda el menor solape. Si ningún eje separa, hay colisión y el eje de menor solape da la normal. *Traza:* rombos de radio 10 con centros (0,0) y (15,0): el eje diagonal tiene solape $5/\sqrt{2} \approx 3{,}5355$, que es el mínimo. Costo ≈ $O((a+b)^2)$ con a y b aristas.

GJK y EPA son otros algoritmos para convexos (se mencionan solo como contexto; no están implementados en el proyecto) [EXTERNO: Ericson, 2004].

B.2 Estructuras de la fase amplia

Técnica	Idea	Costo típico	Fuente
Fuerza bruta	Probar todos los pares: n(n−1)/2	$O(n^2)$	[EXTERNO] Ericson (2004)
Sweep and Prune	Ordenar los intervalos de cada objeto en un eje; solo se comparan los que se solapan	O(n log n + k)	[EXTERNO] Mirtich (1997)
Cuadrícula / hash espacial	Cada objeto cae en una celda ⌊x/c⌋, ⌊y/c⌋; solo comparten pares los de la misma celda	≈ O(n + k)	[EXTERNO] Ericson (2004)
QuadTree	Árbol que divide cada región en 4 donde hay más objetos	≈ O(n log n) si está balanceado	[EXTERNO] Finkel y Bentley (1974)

k es el número de pares que de verdad se solapan. Los costos son típicos, no garantías: dependen de cómo se distribuyen los objetos. Una vista general se encuentra en Lysenko (2015).

Qué usa Godot por dentro. Según el código fuente consultado (rama *master* del repositorio de Godot, que puede ser más reciente que la versión 4.7 del proyecto), el motor 2D por defecto usa una fase amplia con BVH (jerarquía de volúmenes envolventes) y una fase estrecha basada en SAT. Es información de lectura del código fuente, no algo que se haya comprobado ejecutando el motor.

B.3 Detección discreta y continua (diapositiva 9)

En la **discreta** solo se comparan las posiciones de cada cuadro: es barata y simple, pero un objeto rápido o delgado puede cruzar a otro sin solaparse nunca (túnel). En la **continua** se revisa el trayecto entre dos cuadros y se calcula el instante del impacto. Ejemplo del video de referencia: pared a d = 2 m, bala a v = 2 m/s → $t^* = d / v = 1$ s [EXTERNO: video de referencia]. Alternativas más baratas: dividir el paso de física en varios pasos menores, usar paredes más gruesas o lanzar un rayo (raycast). En Godot, `RigidBody2D.continuous_cd` activa la detección continua y por defecto está desactivada (valor 0), según la documentación de la clase. **No se probó en el proyecto:** las pelotas del laboratorio usan el modo por defecto.

B.4 Comparación y aplicaciones (diapositivas 13 y 14)

Técnica	Ventaja	Limitación	Encaja en
AABB / círculo	O(1), muy barata	Ajuste aproximado	Plataformas 2D, filtros
SAT	Exacta; da normal y profundidad	Solo convexos; $O((a+b)^2)$	Obstáculos poligonales
Sweep and Prune	Aprovecha la coherencia entre cuadros	Se degrada si muchos se solapan en el eje	Muchos objetos en movimiento
Cuadrícula	Consulta casi O(1) por celda	Mal con tamaños muy distintos o agrupados	Mundos de bloques, muchas balas
QuadTree / Octree	Se adapta a zonas densas y vacías	Actualización costosa si todo se mueve	Niveles grandes y desiguales
Continua	No hay túnel	Más cara	Proyectiles rápidos

Las cuatro aplicaciones de la diapositiva 14 son descripciones de *cómo se combinan las técnicas*, no afirmaciones sobre el código interno de un juego comercial: plataformas 2D (AABB contra baldosas vecinas), disparos 3D (cajas por parte del cuerpo y rayos o detección continua), mundos de bloques (caja de la entidad contra los bloques que ocupa) y billar o física (círculos/esferas y respuesta por rebote). Los ejemplos visuales (cajas por parte del cuerpo, caja sobre un zombi de bloques) se tomaron de lo que muestra el video; **no pude oír su audio** (no había herramienta de transcripción), así que solo se usó lo visible.

C. Relación con el contenido de la clase

Todo lo marcado [CLASE] sale de las diapositivas del profesor Eduardo David Angulo Madrid. La conexión con colisiones es una **interpretación del grupo**: las diapositivas no hablan de videojuegos.

Tema de clase	Concepto [CLASE]	Cómo funciona	Relación con colisiones
Árboles y ABB (Angulo Madrid, s. f.-a)	Un árbol organiza datos jerárquicamente; en un ABB la altura determina la eficiencia de la búsqueda. Se menciona la idea de partición de espacios binarios.	Cada nodo divide el conjunto en ramas; si el árbol está equilibrado, se descarta la mitad (o más) en cada paso.	El QuadTree es un árbol: cada nodo divide el espacio en 4. Igual que en el ABB, si está desbalanceado (muchos objetos en una rama) pierde eficiencia.
Ordenamientos y búsquedas (s. f.-e)	Análisis de complejidad; comparación O(n) frente a O(log n).	Ordenar una vez permite buscar sin revisar todo.	Sweep and Prune ordena los intervalos en un eje para no comparar todos los pares. El costo $O(n^2)$ de la fuerza bruta se analiza con esta misma idea.
Tablas hash (s. f.-b)	Función hash tipo clave % p; las «colisiones» de hash son dos claves en la misma posición.	La función convierte la clave en una posición del arreglo.	La cuadrícula espacial es un hash de la posición a una celda. **Cuidado con la homonimia:** colisión de hash (dos claves en una celda) no es colisión física (dos objetos solapados).
Grafos (s. f.-c)	G = (V, E); recorridos DFS y BFS.	El recorrido visita los vértices alcanzables desde uno inicial.	Cada cuerpo es un vértice y cada contacto una arista; las componentes conexas son grupos de cuerpos en contacto. **Implementado** en `laboratorio.gd` (`_contar_grupos`, DFS con pila).
Hilos y sockets (s. f.-d)	Dividir un problema grande para resolverlo en paralelo.	Varios hilos trabajan partes distintas.	Solo como idea: la física podría repartirse. **No** está implementado en el proyecto del grupo.

Explicación de la implementación de colisiones en Godot

Estado de verificación (leer primero). Todo lo descrito aquí corresponde a los archivos reales de la carpeta `ejemplo-colision-videojuegos`. **Verificación real:** el grupo ejecutó en Godot 4.7.2-stable (consola, modo headless) las dos órdenes de prueba: `--script res://scripts/pruebas_manuales.gd` dio 17 pruebas y 0 fallos, y `--quit` cargó el proyecto sin errores ni advertencias (ambas con código de salida 0; resultado informado por Claude Code en su equipo). Además se revisó la sintaxis con gdtoolkit y se replicaron las pruebas en Python. **Lo que sigue sin verificar** es la ejecución visual (F5): rebotes, contadores, HUD y monedas no se han observado, porque el modo headless no dibuja la escena. Además, el proyecto original era una plantilla vacía (solo `project.godot` e `icon.svg`): las escenas y scripts son nuevos; se reutilizó la versión (4.7, Forward Plus) y la configuración existentes.

D.1 Qué se modificó en `project.godot`

- Escena principal: `run/main_scene="res://escenas/laboratorio.tscn"`.
- Ventana de 1280×720 (`viewport_width/height`).
- Capas de física con nombre (sección `[layer_names]`): 1 «paredes», 2 «jugador», 3 «pelotas», 4 «monedas».
- Se conservó `config/features=PackedStringArray("4.7", "Forward Plus")` y el resto de la configuración original.

D.2 Estructura de archivos y por qué

Archivo	Nodo raíz	Justificación
escenas/laboratorio.tscn	Node2D «Laboratorio»	Escena principal: reúne todo y la interfaz (CanvasLayer «Interfaz» con Label «Texto»).
escenas/paredes.tscn	StaticBody2D «Paredes»	Cuatro CollisionShape2D (Arriba, Abajo, Izquierda, Derecha) con RectangleShape2D de 1280×20 y 20×720. Un solo cuerpo estático con varias formas.
escenas/obstaculo.tscn	StaticBody2D «Obstaculo»	CollisionPolygon2D «Forma» con triángulo (0,−60), (70,50), (−70,50), rotado 0,261799 rad (15°). Demuestra un polígono convexo.
escenas/jugador.tscn	CharacterBody2D «Jugador»	Cuerpo controlado por código; forma RectangleShape2D de 40×40.
escenas/pelota.tscn	RigidBody2D «Pelota»	Cuerpo con física; forma CircleShape2D de radio 20.
escenas/moneda.tscn	Area2D «Moneda»	Zona de detección; CircleShape2D de radio 16.
scripts/colision_manual.gd	RefCounted (`ColisionManual`)	Pruebas de la fase estrecha escritas a mano, para comparar con el motor.
scripts/pruebas_manuales.gd	SceneTree	17 comprobaciones ejecutables sin abrir el editor.

D.3 Cómo se configura la detección

Escena	collision_layer	collision_mask	Significado
Paredes / Obstaculo	1 (valor por defecto, no se cambió)	1 (valor por defecto)	Están en la capa «paredes». Al ser estáticos no necesitan escanear.
Jugador	2	5 = capas 1 y 3	Está en «jugador»; choca con paredes y pelotas.
Pelota	4 = capa 3	7 = capas 1, 2 y 3	Está en «pelotas»; choca con paredes, jugador y otras pelotas.
Moneda	8 = capa 4	2 = capa 2	Está en «monedas»; solo escanea al jugador.

Los valores guardados en los `.tscn` son **máscaras de bits**: capa 1 = 1, capa 2 = 2, capa 3 = 4, capa 4 = 8. Por eso la pelota (capa 3) guarda `collision_layer = 4` y el jugador, que escanea capas 1

y 3, guarda 1 + 4 = `5`.

ESCENAS/PELOTA.TSCN · CONFIGURACIÓN DEL RIGIDBODY2D

```
12  [node name="Pelota" type="RigidBody2D" groups=["pelotas"]]
13  collision_layer = 4
14  collision_mask = 7
15  physics_material_override = SubResource("PhysicsMaterial_pelota")
16  gravity_scale = 0.0
17  can_sleep = false
18  contact_monitor = true
19  max_contacts_reported = 8
20  linear_damp_mode = 1
21  angular_damp_mode = 1
22  script = ExtResource("1_pelota")
```

Línea a línea: `physics_material_override` aplica un PhysicsMaterial con `bounce = 1.0` y `friction = 0.0` (rebote perfecto, sin roce); `gravity_scale = 0.0` evita que caiga; `can_sleep = false` impide que el motor la «duerma»; `linear_damp_mode = 1` y `angular_damp_mode = 1` activan el modo *Replace*: el amortiguamiento del cuerpo (que aquí queda en su valor por defecto, 0) sustituye al del proyecto, para que la pelota no vaya perdiendo velocidad; `contact_monitor = true` y `max_contacts_reported = 8` son **obligatorios** para que el RigidBody2D emita `body_entered` y permita `get_colliding_bodies()` [EXTERNO: Godot Engine contributors, s. f.-c].

No verificado: el efecto exacto de `linear_damp_mode = 1` (Replace) con amortiguamiento 0 se tomó de la documentación; no se observó con F5 que las pelotas conserven su velocidad.

D.4 Qué ocurre al haber contacto, según cada tipo

Jugador (CharacterBody2D) — respuesta programada.

SCRIPTS/JUGADOR.GD · LÍNEAS 26–47

```
26  func _physics_process(_delta: float) -> void:
27      var direccion: Vector2 = Vector2(
28          Input.get_axis("ui_left", "ui_right"),
29          Input.get_axis("ui_up", "ui_down")
30      )
31      var d_tecla: float = float(Input.is_physical_key_pressed(KEY_D))
32      var a_tecla: float = float(Input.is_physical_key_pressed(KEY_A))
33      var s_tecla: float = float(Input.is_physical_key_pressed(KEY_S))
34      var w_tecla: float = float(Input.is_physical_key_pressed(KEY_W))
35      direccion += Vector2(d_tecla - a_tecla, s_tecla - w_tecla)
36      direccion = direccion.limit_length(1.0)
37
38      velocity = direccion * velocidad
39      move_and_slide()
40
41      # Respuesta a la colisión: revisar con qué se chocó en este paso.
42      _tocando = get_slide_collision_count() > 0
43      if _tocando and not _tocaba_antes:
44          var info: KinematicCollision2D = get_slide_collision(0)
45          choque.emit(info.get_collider(), info.get_normal())
46      _tocaba_antes = _tocando
47      queue_redraw()
```

Línea por línea: (27–29) se lee la dirección con las acciones `ui_left/right/up/down` (flechas); (31–34) se suman las teclas WASD con `Input.is_physical_key_pressed`; (36) `limit_length(1.0)` evita que la diagonal sea más rápida; (38–39) se asigna `velocity` y `move_and_slide()` mueve el cuerpo y lo **desliza** contra lo que choque; (42) `get_slide_collision_count()` dice si hubo contactos en este paso; (43–45) solo cuando el contacto *empieza* (`_tocando` y no `_tocaba_antes`) se lee el primer contacto con `get_slide_collision(0)` y se emite la señal `choque(collider, normal)`. La normal que devuelve Godot es la del contacto, y es la misma noción de «normal» de la diapositiva 7.

Pelota (RigidBody2D) — respuesta del motor. El código solo da la velocidad inicial y escucha el evento; no calcula el rebote.

SCRIPTS/PELOTA.GD · LÍNEAS 22–40

```
22  func _ready() -> void:
23      radio = (_forma.shape as CircleShape2D).radius
24      linear_velocity = velocidad_inicial
25      # contact_monitor y max_contacts_reported están activados en pelota.tscn;
26      # sin ellos el RigidBody2D no emite body_entered.
27      body_entered.connect(_al_chocar)
28
29
30  func _process(delta: float) -> void:
31      if _destello > 0.0:
32          _destello = maxf(0.0, _destello - delta)
33          queue_redraw()
34
35
36  func _al_chocar(cuerpo: Node) -> void:
37      _destello = 0.15
38      rebote.emit(self, cuerpo)
39      queue_redraw()
40
```

En `_ready` se lee el radio de la forma, se asigna `linear_velocity = velocidad_inicial` y se conecta `body_entered`. Cuando la pelota choca, `_al_chocar` activa un destello de 0,15 s y emite `rebote`. **El rebote en sí lo calcula el motor** con el `PhysicsMaterial`.

Moneda (Area2D) — solo detección.

SCRIPTS/MONEDA.GD · LÍNEAS 17–25

```
17  func _ready() -> void:
18      radio = (_forma.shape as CircleShape2D).radius
19      body_entered.connect(_al_entrar_cuerpo)
20
21
22  func _al_entrar_cuerpo(cuerpo: Node2D) -> void:
23      if cuerpo.is_in_group("jugador"):
24          recogida.emit(self)
25          queue_free()
```

El Area2D no bloquea nada. Su máscara (2) solo ve al jugador. Cuando un cuerpo entra, `body_entered` llama a `_al_entrar_cuerpo`; si el cuerpo pertenece al grupo «jugador» se emite `recogida` y la moneda se elimina con `queue_free()`.

D.5 Pruebas manuales de la fase estrecha: `colision_manual.gd`

Funciones estáticas que devuelven un Dictionary: `{}` si no hay colisión; `{"normal", "profundidad"}` si la hay (la normal apunta del primer cuerpo al segundo; en `circulo_aabb` apunta del círculo hacia la caja).

SCRIPTS/COLISION_MANUAL.GD · LÍNEAS 19–50

```
 19  ## Círculo contra círculo: hay colisión si |d|^2 < (ra + rb)^2.
 20  static func circulo_circulo(ca: Vector2, ra: float, cb: Vector2, rb: float) ->
Dictionary:
 21      var d: Vector2 = cb - ca
 22      var suma: float = ra + rb
 23      var dist2: float = d.length_squared()
 24      if dist2 >= suma * suma:
 25          return {}
 26      var dist: float = sqrt(dist2)
 27      var normal: Vector2 = Vector2.RIGHT
 28      if dist > 0.0:
 29          normal = d / dist
 30      return {"normal": normal, "profundidad": suma - dist}
 31
 32
 33  ## Caja contra caja (AABB): los intervalos deben solaparse en X y en Y.
 34  ## El eje con menor solape determina la normal y la profundidad.
 35  static func aabb_aabb(a: Rect2, b: Rect2) -> Dictionary:
 36      var ox: float = minf(a.end.x, b.end.x) - maxf(a.position.x, b.position.x)
 37      var oy: float = minf(a.end.y, b.end.y) - maxf(a.position.y, b.position.y)
 38      if ox <= 0.0 or oy <= 0.0:
 39          return {}
 40      var centro_a: Vector2 = a.get_center()
 41      var centro_b: Vector2 = b.get_center()
 42      if ox < oy:
 43          var signo_x: float = 1.0
 44          if centro_b.x < centro_a.x:
 45              signo_x = -1.0
 46          return {"normal": Vector2(signo_x, 0.0), "profundidad": ox}
 47      var signo_y: float = 1.0
 48      if centro_b.y < centro_a.y:
 49          signo_y = -1.0
 50      return {"normal": Vector2(0.0, signo_y), "profundidad": oy}
```

Círculo–círculo (20–30): compara el cuadrado de la distancia con el cuadrado de la suma de radios (24); si `dist2 >= suma²` devuelve `{}`. Si los centros coinciden (`dist == 0`) usa la normal `Vector2.RIGHT` para evitar dividir entre cero. AABB–AABB (35–50): calcula `ox` y `oy`; si alguno es ≤ 0 no hay colisión; el menor define la normal, con el signo según qué centro esté a la izquierda o arriba.

SCRIPTS/COLISION_MANUAL.GD · LÍNEAS 56–83

```
 56  static func circulo_aabb(centro: Vector2, radio: float, caja: Rect2) ->
Dictionary:
 57      var p: Vector2 = Vector2(
 58          clampf(centro.x, caja.position.x, caja.end.x),
 59          clampf(centro.y, caja.position.y, caja.end.y)
 60      )
 61      var d: Vector2 = centro - p
 62      var dist2: float = d.length_squared()
 63      if dist2 == 0.0:
 64          # El centro está dentro de la caja: se sale por la cara más cercana.
 65          var izq: float = centro.x - caja.position.x
 66          var der: float = caja.end.x - centro.x
 67          var arr: float = centro.y - caja.position.y
 68          var aba: float = caja.end.y - centro.y
 69          var menor: float = minf(minf(izq, der), minf(arr, aba))
 70          var normal: Vector2 = Vector2.DOWN
 71          if menor == izq:
 72              normal = Vector2.RIGHT
 73          elif menor == der:
 74              normal = Vector2.LEFT
 75          elif menor == arr:
 76              normal = Vector2.DOWN
 77          else:
 78              normal = Vector2.UP
 79          return {"normal": normal, "profundidad": menor + radio}
 80      if dist2 >= radio * radio:
 81          return {}
 82      var dist: float = sqrt(dist2)
 83      return {"normal": -d / dist, "profundidad": radio - dist}
```

Círculo–AABB: `clampf` recorta el centro al rectángulo para obtener P (57–60); `d = centro − P` (61). Si `dist2 == 0` el centro está dentro (63–79): se calcula la distancia a cada cara y se sale por la menor; profundidad = menor + radio. Si no, compara contra el radio al cuadrado (80) y devuelve normal `−d/dist` y profundidad `radio − dist`.

SCRIPTS/COLISION_MANUAL.GD · LÍNEAS 89–107

```
 89  static func sat_poligonos(va: PackedVector2Array, vb: PackedVector2Array) ->
Dictionary:
 90      var menor: float = INF
 91      var eje_menor: Vector2 = Vector2.ZERO
 92      for poligono in [va, vb]:
 93          var n: int = poligono.size()
 94          for i in n:
 95              var arista: Vector2 = poligono[(i + 1) % n] - poligono[i]
 96              var eje: Vector2 = Vector2(-arista.y, arista.x).normalized()
 97              var pa: Vector2 = _proyectar(va, eje)
 98              var pb: Vector2 = _proyectar(vb, eje)
 99              var solape: float = minf(pa.y, pb.y) - maxf(pa.x, pb.x)
100              if solape <= 0.0:
101                  return {}
102              if solape < menor:
103                  menor = solape
104                  eje_menor = eje
105      if (_centro(vb) - _centro(va)).dot(eje_menor) < 0.0:
106          eje_menor = -eje_menor
107      return {"normal": eje_menor, "profundidad": menor}
```

SAT: recorre las aristas de ambos polígonos (92–94); el eje es la perpendicular normalizada (96); proyecta ambos (97–98) y calcula el solape (99). Si ≤ 0 hay un eje separador y devuelve `{}` (salida temprana, 100–101); si no, guarda el menor (102–104). Al final orienta la normal de A hacia B (105–106).

Cómo se verificó: `pruebas_manuales.gd` define 17 comprobaciones (por ejemplo círculos con radio 10 y centros a 15 → profundidad 5; rombos radio 10 a 15 → profundidad 3,5355; cuadrados → mismo resultado que AABB). Se ejecutan con `godot --headless --path . --script res://scripts/pruebas_manuales.gd`. Esa orden se ejecutó en Godot 4.7.2-stable: **17 pruebas, 0 fallos**. Antes se habían comprobado los mismos casos en Python. Esto valida la matemática de `colision_manual.gd` dentro de Godot, pero no el comportamiento visual de la escena.

D.6 El laboratorio: contadores, AABB, grafo de contactos y comparación manual/motor

SCRIPTS/LABORATORIO.GD · LÍNEAS 29–36

```
 29  func _ready() -> void:
 30      _jugador.choque.connect(_al_chocar_jugador)
 31      for pelota: Pelota in get_tree().get_nodes_in_group("pelotas"):
 32          pelota.rebote.connect(_al_rebotar)
 33      for moneda: Moneda in $Monedas.get_children():
 34          moneda.recogida.connect(_al_recoger)
 35          _monedas_total += 1
 36      _actualizar_texto()
```

`_ready` conecta las señales: `choque` del jugador, `rebote` de cada pelota (se obtienen del grupo «pelotas») y `recogida` de cada moneda. Así el laboratorio solo *cuenta* los eventos; no vuelve a

calcular colisiones. Esto sigue la idea de desacoplar con señales.

SCRIPTS/LABORATORIO.GD · LÍNEAS 81–98

```
 81  ## Componentes conexas del grafo de contactos (recorrido DFS con pila).
 82  func _contar_grupos() -> int:
 83      var visitados: Dictionary = {}
 84      var grupos: int = 0
 85      for inicio in _cuerpos_del_grafo():
 86          if visitados.has(inicio):
 87              continue
 88          grupos += 1
 89          var pila: Array = [inicio]
 90          while not pila.is_empty():
 91              var actual: Node = pila.pop_back()
 92              if visitados.has(actual):
 93                  continue
 94              visitados[actual] = true
 95              for vecino in _vecinos(actual):
 96                  if not visitados.has(vecino):
 97                      pila.append(vecino)
 98      return grupos
```

Grafo de contactos: cada cuerpo es un vértice, los vecinos son los cuerpos con los que está en contacto (`get_colliding_bodies()`), y `_contar_grupos` cuenta las **componentes conexas** con un DFS iterativo con pila (`pila.pop_back()` , `visitados` como Dictionary). Es la aplicación directa del tema de Grafos de la clase.

SCRIPTS/LABORATORIO.GD · LÍNEAS 46–57

```
 46  func _physics_process(_delta: float) -> void:
 47      _grupos = _contar_grupos()
 48      var pelotas: Array = get_tree().get_nodes_in_group("pelotas")
 49      if not pelotas.is_empty():
 50          var primera: Pelota = pelotas[0]
 51          # Prueba manual (explicada en la exposición) frente al resultado del
motor.
 52          var manual: Dictionary = ColisionManual.circulo_aabb(
 53              primera.global_position, primera.radio + MARGEN_CONTACTO,
_jugador.caja_global()
 54          )
 55          _manual_pelota1 = not manual.is_empty()
 56          _motor_pelota1 = primera.get_colliding_bodies().has(_jugador)
 57      _actualizar_texto()
```

Comparación manual/motor: para la primera pelota se calcula `ColisionManual.circulo_aabb` con el radio + `MARGEN_CONTACTO` (2 px) contra la caja del jugador, y se compara con `get_colliding_bodies().has(_jugador)` . El margen existe porque el motor separa los cuerpos al resolver el contacto, de modo que casi nunca quedan solapados. El resultado se muestra en el HUD («prueba manual: sí/no | motor: sí/no»). **No verificado visualmente:** con qué frecuencia coinciden ambos valores en el HUD.

La tecla Espacio activa/desactiva el dibujo de las AABB (cian) y de las líneas de contacto (rosa), dibujadas por `superposicion.gd` sobre el nodo «Superposicion» (`z_index = 10`); Esc recarga la escena.

D.7 Relación con la teoría

Teoría (diapositiva)	Dónde está en el proyecto
Detección vs. respuesta (4, 5)	Area2D detecta (Moneda); RigidBody2D responde el motor (Pelota); CharacterBody2D responde el código (Jugador).
Formas (6)	RectangleShape2D, CircleShape2D, CollisionPolygon2D (triángulo convexo del obstáculo).
Fase estrecha (7, 8)	`colision_manual.gd` : círculo–círculo, AABB–AABB, círculo–AABB y SAT.
AABB como filtro (11)	Cajas cian dibujadas en el laboratorio (`caja_global`).
Grafos (12)	`_contar_grupos` con DFS.
Capas y máscaras (16)	`[layer_names]` y los valores de `collision_layer/mask` en los .tscn.
Fase amplia (11)	**No** se implementó QuadTree, sweep and prune ni cuadrícula en el proyecto: se explican como teoría. Godot ya trae su propia fase amplia.

Evidencia pendiente. Falta una captura real de la ejecución con F5. Para completar la diapositiva 18: abrir el proyecto en Godot 4.7.2, pulsar F5, esperar unos segundos y tomar la captura; y guardar una captura de la terminal con «Pruebas: 17 | fallos: 0».

E. Preguntas probables del profesor

E.1 Conceptuales

1. [Conceptual] ¿Cuál es la diferencia entre detectar y responder a una colisión?
Respuesta: Detectar es saber si hay contacto y medirlo; responder es decidir qué hace el juego.
Explicación: La detección produce normal y profundidad; la respuesta los usa (separar, rebotar, emitir un evento).
Ejemplo: La moneda detecta al jugador; la pelota rebota.

2. [Conceptual] ¿Por qué se usan dos fases?

Respuesta: Probar con exactitud todos los pares cuesta $O(n^2)$; la fase amplia descarta casi todos con un filtro barato.
Explicación: Con n = 1.000 hay 499.500 pares por cuadro; en 16,7 ms no caben pruebas exactas de todos.
Ejemplo: Dos objetos en esquinas opuestas se descartan por sus AABB sin ejecutar SAT.

3. [Conceptual] ¿Qué es el túnel y cómo se evita?

Respuesta: Que un objeto rápido atraviese otro porque en ningún cuadro se solapan.
Explicación: La detección discreta solo mira posiciones por cuadro. Se evita con detección continua (CCD), paredes más gruesas o menor velocidad [EXTERNO].
Ejemplo: Una bala que está antes y después de una pared fina.

4. [Conceptual] ¿Por qué cuanto más simple la forma, mejor?

Respuesta: Porque la prueba es más barata y más fácil de acertar.
Explicación: Círculo y AABB cuestan $O(1)$; SAT cuesta $\approx O((a+b)^2)$.
Ejemplo: Un personaje se envuelve en un rectángulo, no en su silueta exacta.

5. [Conceptual] ¿Qué dice el teorema del eje separador?

Respuesta: Dos convexos no se solapan si y solo si existe un eje donde sus proyecciones están separadas.
Explicación: Basta un eje con hueco para descartar; si ninguno lo tiene, hay colisión. Solo vale para convexos.
Ejemplo: Rombos a distancia 25: el eje diagonal ya tiene hueco.

6. [Conceptual] ¿Qué diferencia hay entre colisión física y colisión de hash?

Respuesta: La física es solape de objetos; la de hash es que dos claves caen en la misma posición de la tabla [CLASE].
Explicación: Comparten nombre pero son problemas distintos.
Ejemplo: En la cuadrícula, dos objetos en la misma celda son candidatos, no necesariamente chocan.

E.2 Técnicas y estructuras

7. [Técnico] ¿Por qué la complejidad del QuadTree es «≈ O(n log n)» y no siempre?

Respuesta: Depende del balance: si los objetos se concentran, el árbol se desbalancea.
Explicación: Igual que un ABB, la altura determina la eficiencia [CLASE]. Los costos de la diapositiva 11 son típicos, no garantizados.
Ejemplo: Todos los objetos en una esquina producen un árbol profundo.

8. [Técnico] ¿Qué estructura elegiría para un juego con objetos de tamaño parecido?

Respuesta: Una cuadrícula (hash espacial).
Explicación: Con celdas del tamaño de los objetos el costo es ≈ O(n + k) y es simple de implementar [EXTERNO: Ericson, 2004].
Ejemplo: Mapa de azulejos con muchas pelotas pequeñas.

9. [Técnico] ¿Cómo se calcula la profundidad entre dos círculos?

Respuesta: (rA + rB) − |d|.
Explicación: Es cuánto se meten uno en el otro; con la normal d/|d| forma el MTV.
Ejemplo: Radios 10 y 10, distancia 15: profundidad 5.

10. [Técnico] ¿Cómo conecta el tema con grafos?

Respuesta: Cuerpos = vértices; contactos = aristas; grupos = componentes conexas.
Explicación: Un DFS visita todo lo alcanzable; cada inicio nuevo es un grupo nuevo [CLASE: DFS/BFS].
Ejemplo: `_contar_grupos` en `laboratorio.gd`.

11. [Técnico] ¿Qué diferencia hay entre SAT y GJK?

Respuesta: Ambos sirven para convexos; SAT prueba ejes, GJK trabaja con la diferencia de Minkowski.
Explicación: Solo SAT se explicó e implementó; GJK es contexto [EXTERNO: Ericson, 2004].
Ejemplo: —

E.3 Sobre Godot y el proyecto

12. [Godot] ¿Cuándo uso Area2D, StaticBody2D, RigidBody2D y CharacterBody2D?

Respuesta: Area2D: detectar sin bloquear. Static: obstáculos fijos. Rigid: física simulada por el motor. Character: control por código.
Explicación: Cada uno cubre una necesidad distinta [EXTERNO: Godot Engine contributors, s. f.-b].
Ejemplo: Moneda, paredes, pelota, jugador.

13. [Godot] ¿Para qué sirven collision_layer y collision_mask?

Respuesta: Layer: en qué capa estoy. Mask: qué capas escaneo.
Explicación: Evitan pruebas innecesarias y permiten reglas como «las pelotas atraviesan la moneda».
Ejemplo: Moneda: mask 2 (solo jugador).

14. [Godot] ¿Por qué la pelota no emitía body_entered si olvidamos algo?

Respuesta: Porque falta `contact_monitor` y `max_contacts_reported`.
Explicación: Sin ellos el RigidBody2D no reporta contactos [EXTERNO: Godot Engine contributors, s. f.-c].
Ejemplo: Están activados en `pelota.tscn` (true y 8).

15. [Godot] ¿Dónde está el código de rebote de la pelota?

Respuesta: No hay: lo hace el motor con el PhysicsMaterial.
Explicación: `bounce = 1.0` y `friction = 0.0` en `pelota.tscn` y en las paredes/obstáculo.
Ejemplo: Solo se da `linear_velocity` inicial.

16. [Godot] ¿Qué hace move_and_slide y por qué el jugador es CharacterBody2D?

Respuesta: Mueve el cuerpo con `velocity` y lo desliza contra lo que choca.
Explicación: El CharacterBody2D no tiene física propia; la respuesta la define el código [EXTERNO: Godot Engine contributors, s. f.-e].
Ejemplo: `jugador.gd` líneas 38–39.

17. [Godot] ¿Qué algoritmos usa internamente Godot?

Respuesta: Por lectura del código fuente: fase amplia con BVH y fase estrecha con SAT.
Explicación: Consultado en la rama *master*; puede diferir de la 4.7 y no se comprobó ejecutando.
Ejemplo: Archivos `godot_broad_phase_2d_bvh.h` y `godot_collision_solver_2d_sat.cpp`.

18. [Godot] ¿Cómo comprobaron que su implementación funciona?

Respuesta: Con 17 comprobaciones numéricas ejecutadas en Godot 4.7.2 (headless, 0 fallos), carga del proyecto sin errores y revisión de sintaxis; la ejecución visual con F5 la mostramos en la demostración.
Explicación: Hay que decirlo con claridad y mostrar luego la ejecución real.
Ejemplo: Orden: `godot --headless --path . --script res://scripts/pruebas_manuales.gd`.

19. [Godot] ¿Por qué no implementaron un QuadTree en Godot?

Respuesta: Porque Godot ya incluye su fase amplia; el laboratorio se centró en los cuatro cuerpos, las capas y la fase estrecha manual.
Explicación: Implementar uno propio duplicaría el motor.
Ejemplo: La estructura se explica en teoría y se relaciona con la clase.

Referencias (IEEE, mismo orden que la presentación)

Los números coinciden con los de la presentación. La rúbrica exige IEEE. [CLASE] = material de curso; el video no tiene autor ni URL verificados.

[1] C. Ericson, *Real-Time Collision Detection*. San Francisco, CA, EE. UU.: Morgan Kaufmann, 2004.

[2] Godot Engine contributors, «Physics introduction», *Godot Docs*, s. f. [En línea]. Disponible: https://docs.godotengine.org/en/stable/tutorials/physics/physics_introduction.html

[3] «How Collisions Work in Games», video explicativo, 5 min 53 s. Archivo proporcionado por el grupo; autor y URL no verificados.

[4] Godot Engine contributors, «Collision shapes (2D)», *Godot Docs*, s. f. [En línea]. Disponible: https://docs.godotengine.org/en/stable/tutorials/physics/collision_shapes_2d.html

[5] Kansas State University, «Separating axis theorem», *CIS 580 Textbook*, s. f. [En línea]. Disponible: https://textbooks.cs.ksu.edu/cis580/04-collisions/04-separating-axis-theorem/

[6] Godot Engine contributors, «RigidBody2D», *Godot Docs*, s. f. [En línea]. Disponible: https://docs.godotengine.org/en/stable/classes/class_rigidbody2d.html

[7] M. Lysenko, «Collision detection (part 1): Overview», *0 FPS*, ene. 2015. [En línea]. Disponible: https://0fps.net/2015/01/07/collision-detection-part-1/

[8] E. D. Angulo Madrid, «Ordenamientos y búsquedas», diapositivas de clase, Estructura de Datos II, Universidad del Norte, s. f.

[9] B. Mirtich, «Efficient algorithms for two-phase collision detection», Mitsubishi Electric Research Laboratories, Informe técnico TR-97-23, dic. 1997. [En línea]. Disponible: https://merl.com/publications/docs/TR97-23.pdf

[10] R. A. Finkel y J. L. Bentley, «Quad trees: A data structure for retrieval on composite keys», *Acta Informatica*, vol. 4, n.° 1, pp. 1–9, 1974, doi: 10.1007/BF00288933.

[11] E. D. Angulo Madrid, «Árboles», diapositivas de clase, Estructura de Datos II, Universidad del Norte, s. f.

[12] E. D. Angulo Madrid, «Búsqueda y organización de la información», diapositivas de clase, Estructura de Datos II, Universidad del Norte, s. f.

[13] E. D. Angulo Madrid, «Grafos», diapositivas de clase, Estructura de Datos II, Universidad del Norte, s. f.

[14] E. D. Angulo Madrid, «Hilos y sockets», diapositivas de clase, Estructura de Datos II, Universidad del Norte, s. f.

[15] Godot Engine contributors, «Using CharacterBody2D», *Godot Docs*, s. f. [En línea]. Disponible: https://docs.godotengine.org/en/stable/tutorials/physics/using_character_body_2d.html

[16] Godot Engine contributors, «Using Area2D», *Godot Docs*, s. f. [En línea]. Disponible: https://docs.godotengine.org/en/stable/tutorials/physics/using_area_2d.html

Anexo: formato APA 7 (por si se solicita)

Angulo Madrid, E. D. (s. f.-a). *Árboles* [Diapositivas de clase]. Estructura de Datos II, Universidad del Norte.

Angulo Madrid, E. D. (s. f.-b). *Búsqueda y organización de la información* [Diapositivas de clase]. Estructura de Datos II, Universidad del Norte.

Angulo Madrid, E. D. (s. f.-c). *Grafos* [Diapositivas de clase]. Estructura de Datos II, Universidad del Norte.

Angulo Madrid, E. D. (s. f.-d). *Hilos y sockets* [Diapositivas de clase]. Estructura de Datos II, Universidad del Norte.

Angulo Madrid, E. D. (s. f.-e). *Ordenamientos y búsquedas* [Diapositivas de clase]. Estructura de Datos II, Universidad del Norte.

Ericson, C. (2004). *Real-time collision detection*. Morgan Kaufmann.

Finkel, R. A., y Bentley, J. L. (1974). Quad trees: A data structure for retrieval on composite keys. *Acta Informatica*, *4*(1), 1–9. https://doi.org/10.1007/BF00288933

Godot Engine contributors. (s. f.-a). *Collision shapes (2D)*. Godot Docs. https://docs.godotengine.org/en/stable/tutorials/physics/collision_shapes_2d.html

Godot Engine contributors. (s. f.-b). *Physics introduction*. Godot Docs. https://docs.godotengine.org/en/stable/tutorials/physics/physics_introduction.html

Godot Engine contributors. (s. f.-c). *RigidBody2D*. Godot Docs. https://docs.godotengine.org/en/stable/classes/class_rigidbody2d.html

Godot Engine contributors. (s. f.-d). *Using Area2D*. Godot Docs. https://docs.godotengine.org/en/stable/tutorials/physics/using_area_2d.html

Godot Engine contributors. (s. f.-e). *Using CharacterBody2D*. Godot Docs. https://docs.godotengine.org/en/stable/tutorials/physics/using_character_body_2d.html

Kansas State University. (s. f.). *Separating axis theorem*. En *CIS 580 textbook*. https://textbooks.cs.ksu.edu/cis580/04-collisions/04-separating-axis-theorem/

Lysenko, M. (2015, 7 de enero). *Collision detection (part 1): Overview*. 0 FPS. https://0fps.net/2015/01/07/collision-detection-part-1/

Mirtich, B. (1997). *Efficient algorithms for two-phase collision detection* (Informe técnico TR-97-23). Mitsubishi Electric Research Laboratories. https://merl.com/publications/docs/TR97-23.pdf

Autoría y uso de IA

El código y los textos se elaboraron con asistencia de IA (Claude, Anthropic) a partir de las instrucciones del grupo. **Pendiente:** que cada integrante anote qué partes revisó, adaptó o modificó (criterio de declaración de fuentes de la rúbrica).